

Codex Retest Lecture

CS 374 — Algorithms & Models of Computation · Sep 30, 2026 at 11:58 AM · 8:21

Summary

The parser consumes the lexer's tokens and builds a syntax tree from a context-free grammar; today's goal is building an LL(1) parse table by hand. Top-down parsers expand the leftmost nonterminal; recursive descent picks a production by peeking at the next token, and LL(1) means one token always suffices.

Key terms: context-free grammar, predictive parsing, lookahead, recursive descent, left recursion, epsilon production, FIRST set, fixed point

Takeaways

0:00–0:17 From tokens to syntax trees

The parser consumes the lexer's tokens and builds a syntax tree from a context-free grammar; today's goal is building an LL(1) parse table by hand.

- A grammar $G = (N, \Sigma, P, S)$: nonterminals, terminals, productions, start symbol.
- Lexing and parsing are separate phases; the parser never sees raw characters.
- Predictive parsing is the top-down technique covered in this lecture.

Slides 1, 2

0:17–0:33 Predictive parsing chooses productions by lookahead

Top-down parsers expand the leftmost nonterminal; recursive descent picks a production by peeking at the next token, and LL(1) means one token always suffices.

- Recursive descent: one function per nonterminal, each matching one production.
- LL(1) = Left-to-right scan, Leftmost derivation, 1 token of lookahead.
- No backtracking: the lookahead must uniquely determine the production.

Slides 3, 4

0:33–0:53 Eliminate left recursion before parsing top-down

$A \rightarrow A\alpha \mid \beta$ loops forever in recursive descent; rewrite it as $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$, e.g. $E \rightarrow T E'$, $E' \rightarrow + T E' \mid \epsilon$.

- Left recursion: a nonterminal that derives a string starting with itself.
- The rewrite preserves the language but produces right-recursive productions.
- Indirect left recursion ($A \rightarrow Bx$, $B \rightarrow Ay$) must be handled by substitution first.

Slides 5, 6

0:53–1:12 FIRST sets: what a string can start with

$\text{FIRST}(\alpha)$ holds every terminal that can begin a string derived from α , plus ϵ if α can vanish; compute it by iterating the rules to a fixed point.

- $\text{FIRST}(a) = \{a\}$ for any terminal a .
- For $A \rightarrow X_1 \dots X_n$, add $\text{FIRST}(X_1)$; keep going to X_2 only if $\epsilon \in \text{FIRST}(X_1)$.
- Iterate over all productions until no FIRST set changes.
- Example: $\text{FIRST}(E') = \{+, \epsilon\}$; $\text{FIRST}(T) = \text{FIRST}(F) = \{(\text{, id})\}$.

Slides 7, 8

1:12–1:42 FOLLOW sets: what can come after a nonterminal

$\text{FOLLOW}(A)$ is every terminal that can appear right after A ($\$$ for the start symbol); it decides when expanding A to ϵ is safe.

- $B \rightarrow \alpha A \beta$: add $\text{FIRST}(\beta) - \{\epsilon\}$ to $\text{FOLLOW}(A)$.
- If β is empty or $\epsilon \in \text{FIRST}(\beta)$: add $\text{FOLLOW}(B)$ to $\text{FOLLOW}(A)$.
- $\$ \in \text{FOLLOW}(S)$ marks end of input.
- Example: $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{ \text{), } \$ \}$.

Slides 9, 10

1:42–2:00 The LL(1) table and the LL(1) condition

$M[A, a] = \alpha$ for $a \in \text{FIRST}(\alpha)$, and for $b \in \text{FOLLOW}(A)$ when $\epsilon \in \text{FIRST}(\alpha)$; the grammar is LL(1) iff no cell holds two productions.

- Table M is indexed by (nonterminal, terminal or $\$$).
- A conflict (two entries in one cell) means the grammar is not LL(1).
- Disjointness test: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$, and $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ when $\beta \Rightarrow^* \epsilon$.

Slides 11, 12

2:00–2:18 Table-driven LL(1) parsing with a stack

Start the stack with $S \$$; match terminals against the lookahead, replace nonterminals with $M[A, \text{lookahead}]$ reversed, accept when both hit $\$$.

- Terminal on top: must equal lookahead, else syntax error.
- Nonterminal on top: consult the table; empty cell means error.
- The stack contents are always the unmatched suffix of the current sentential form.

Slides 13, 14

2:18–2:41 Left factoring, and where LL(1) runs out

$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ becomes $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ so one token can decide; the dangling else stays ambiguous and some languages aren't LL(k) at all.

- Left factoring delays the decision until enough input distinguishes the alternatives.
- if/then/else needs factoring; the dangling else is resolved by matching the nearest if.
- LL(1) is fast and simple but strictly weaker than LR parsing.

Slides 15, 16

2:41–3:12 Panic-mode error recovery uses FOLLOW as the sync set

On an empty table cell, skip input until a synchronizing token ($\text{FOLLOW}(A)$ works well), pop A , report once and keep parsing; next lecture is bottom-up parsing.

- Panic mode: discard tokens until a synchronizing token appears.
- Using $\text{FOLLOW}(A)$ lets the parser pretend A was completed.
- Avoid cascading errors: report once per recovery.
- Reading: Dragon Book §4.4–4.5 for LR parsing.

Slides 17, 18

Quiz

0:36 Skipped — In LL(1), what does the "1" stand for?

2:09 Not quite — A grammar is LL(1) exactly when...

Transcript

0:00 Okay, uh, let's get started. Before I begin, a small correction to last week's slides: on slide fourteen the identifier token should be id, not ident, so I'll post the corrected slides on the website tonight. So last week we finished the lexer, right, so we can turn a stream of characters into a stream of tokens.

0:08 Today we start on the parser, which takes those tokens and builds a syntax tree according to a context-free grammar. Remember a grammar has four parts: nonterminals, terminals, productions, and a start symbol. Any questions on that before I start today's lecture? Okay. The specific technique we'll cover today is called predictive parsing, and by the end you'll be able to build an LL(1) parse table by hand.

0:17 Top-down parsing starts at the start symbol and repeatedly expands the leftmost nonterminal until the whole input is matched. The simplest implementation is recursive descent: you write one function per nonterminal, and each function tries to match one of its productions. The hard part is the choice. When a nonterminal has, like, several productions, which one do you expand? Anybody?

0:26 Predictive parsers answer that by peeking at the next input token, which we call the lookahead. If one token of lookahead is always enough to decide, the grammar is LL(1): left to right scan, leftmost derivation, one token of lookahead. Is this clear? Okay. Before we can do any of this, we have to deal with left recursion.

0:35 Take E goes to E plus T, or T. So a recursive descent function for E would immediately call itself on E, without consuming any input, and it just loops forever, right. So we rewrite. The general rule is: if A goes to A alpha or beta, replace it with A goes to beta A prime, and A prime goes to alpha A prime or epsilon.

0:44 For our expression grammar that gives E goes to T E prime, and E prime goes to, uh, plus T E prime, or epsilon. And wait, on the slide it says plus E prime, that should be plus T E prime, small mistake, I'll fix it. Same language, but now every production consumes a token before recursing, so the parser terminates.

0:53 Now the two sets that make prediction possible. FIRST of alpha is the set of terminals that can begin some string derived from alpha. If alpha can derive the empty string, we also put epsilon into FIRST of alpha. For a terminal, FIRST is just the terminal itself. For a production A goes to X one through X n, you add FIRST of X one, and if that contains epsilon, you continue with X two, and so on.

1:04 You compute this by iterating: apply the rules to every production, and repeat until no set changes. That's a fixed point. Is this clear so far? Any doubts? For our grammar, FIRST of E prime is plus and epsilon, and FIRST of T is the same as FIRST of F, which is open paren and id. FIRST tells us what a nonterminal can start with. FOLLOW tells us what can come immediately after it.

1:15 FOLLOW of A is the set of terminals that can appear right after A in some sentential form, and we add the end marker, dollar, to FOLLOW of the start symbol. You need FOLLOW precisely when A can derive epsilon, because then the parser has to know whether it's okay to expand A to nothing.

1:22 Sorry, quick question. Is FOLLOW of A just the FIRST set of whatever comes after A?

1:25 Good question. Almost. It's the union of FIRST of everything that can come after A in any production, minus epsilon, and if that can be empty you also pull in FOLLOW of the left-hand side. So it's bigger than any single FIRST set. The rules: for B goes to alpha A beta, add FIRST of beta minus epsilon to FOLLOW of A. And if beta can be empty, or is empty, add FOLLOW of B to FOLLOW of A.

1:36 Again, iterate to a fixed point. For our grammar, FOLLOW of E is close paren and dollar, and FOLLOW of E prime is, uh, the same. People get this wrong on the exam every year, so please practice it. With FIRST and FOLLOW we can build the parse table. Rows are nonterminals, columns are terminals plus the end marker.

1:45 For each production A goes to alpha, put alpha in cell M of A comma a, for every terminal a in FIRST of alpha. If epsilon is in FIRST of alpha, also put alpha in M of A comma b, for every b in FOLLOW of A. Okay so here's the key definition, this is important, this will be on the midterm: a grammar is LL(1) if and only if no cell ends up with two productions.

1:55 Equivalently, for any two alternatives A goes to alpha or beta, their FIRST sets must be disjoint, and if beta can be empty, FIRST of alpha must not overlap FOLLOW of A. Once you have the table, the parser itself is trivial. You keep a stack that starts with the start symbol and the end marker. If the top of the stack is a terminal, it must match the lookahead: pop it and advance the input.

2:06 If the top is a nonterminal A, look up M of A comma lookahead, pop A, and push the right-hand side in reverse. If the cell is empty, that's a syntax error. If the stack and the input are both at dollar, you accept. Everybody with me? Okay. Let's trace id plus id times id. Stack starts E dollar; lookahead is id, so we replace E with T E prime, then T with F T prime, then F with id, match, and keep going.

2:18 Okay, one more transformation you'll need, uh, left factoring. Suppose A goes to alpha beta one, or alpha beta two, with the same prefix alpha. One token of lookahead can't tell those apart, so we factor the prefix: A goes to alpha A prime, and A prime goes to beta one or beta two.

2:26 The classic example is if then else. Both productions start with if expr then stmt, so we factor that out and decide about the else later.

2:30 So for the dangling else, does the factoring actually fix the ambiguity?

2:32 No, and that's exactly the point. Factoring only delays the decision. The dangling else is genuinely ambiguous, and no amount of factoring makes it LL(1), so most compilers resolve it by always matching the nearest if. That's a general lesson. LL(1) is simple and fast, but some languages aren't LL(k) for any k, which is why we'll look at bottom-up parsing next.

2:41 Last thing, and then we're done: what happens when the table cell is empty and you hit an error. You don't want to stop at the first mistake, right. The simplest strategy is panic mode: discard input tokens until you reach a synchronizing token, then pop the nonterminal and continue.

2:48 A good choice of synchronizing set for A is FOLLOW of A, because after skipping to something in FOLLOW of A you can pretend A was parsed and move on. Report the error once, don't cascade, and keep parsing so the user sees more than one problem per compile. Next time we'll do bottom-up parsing, shift-reduce, and LR tables. Read Dragon Book sections four point four and four point five.

3:12 Okay, uh, let's get started. Before I begin, a small correction to last week's slides: on slide fourteen the identifier token should be id, not ident, so I'll post the corrected slides on the website tonight. So last week we finished the lexer, right, so we can turn a stream of characters into a stream of tokens.

3:20 Today we start on the parser, which takes those tokens and builds a syntax tree according to a context-free grammar. Remember a grammar has four parts: nonterminals, terminals, productions, and a start symbol. Any questions on that before I start today's lecture? Okay. The specific technique we'll cover today is called predictive parsing, and by the end you'll be able to build an LL(1) parse table by hand.

3:30 Top-down parsing starts at the start symbol and repeatedly expands the leftmost nonterminal until the whole input is matched. The simplest implementation is recursive descent: you write one function per nonterminal, and each function tries to match one of its productions. The hard part is the choice. When a nonterminal has, like, several productions, which one do you expand? Anybody?

3:38 Predictive parsers answer that by peeking at the next input token, which we call the lookahead. If one token of lookahead is always enough to decide, the grammar is LL(1): left to right scan, leftmost derivation, one token of lookahead. Is this clear? Okay. Before we can do any of this, we have to deal with left recursion.

3:47 Take $E \rightarrow E + T$, or T . So a recursive descent function for E would immediately call itself on E , without consuming any input, and it just loops forever, right. So we rewrite. The general rule is: if $A \rightarrow A\alpha$ or β , replace it with $A \rightarrow \beta A'$, and A' goes to $\alpha A'$ or ϵ .

3:56 For our expression grammar that gives $E \rightarrow T E'$, and $E' \rightarrow + T E'$, or ϵ . And wait, on the slide it says $+ E'$, that should be $+ T E'$, small mistake, I'll fix it. Same language, but now every production consumes a token before recursing, so the parser terminates.

4:05 Now the two sets that make prediction possible. FIRST of α is the set of terminals that can begin some string derived from α . If α can derive the empty string, we also put ϵ into FIRST of α . For a terminal, FIRST is just the terminal itself. For a production $A \rightarrow X_1 \dots X_n$, you add FIRST of X_1 , and if that contains ϵ , you continue with X_2 , and so on.

4:16 You compute this by iterating: apply the rules to every production, and repeat until no set changes. That's a fixed point. Is this clear so far? Any doubts? For our grammar, FIRST of E' is $+$ and ϵ , and FIRST of T is the same as FIRST of F , which is $($ and id . FIRST tells us what a nonterminal can start with. FOLLOW tells us what can come immediately after it.

4:27 FOLLOW of A is the set of terminals that can appear right after A in some sentential form, and we add the end marker, $\$$, to FOLLOW of the start symbol. You need FOLLOW precisely when A can derive ϵ , because then the parser has to know whether it's okay to expand A to nothing.

4:35 Sorry, quick question. Is FOLLOW of A just the FIRST set of whatever comes after A ?

4:37 Good question. Almost. It's the union of FIRST of everything that can come after A in any production, minus ϵ , and if that can be empty you also pull in FOLLOW of the left-hand side. So it's bigger than any single FIRST set. The rules: for $B \rightarrow \alpha A \beta$, add FIRST of β minus ϵ to FOLLOW of A . And if β can be empty, or is empty, add FOLLOW of B to FOLLOW of A .

4:48 Again, iterate to a fixed point. For our grammar, FOLLOW of E is $)$ and $\$$, and FOLLOW of E' is, uh, the same. People get this wrong on the exam every year, so please practice it. With FIRST and FOLLOW we can build the parse table. Rows are nonterminals, columns are terminals plus the end marker.

4:56 For each production $A \rightarrow \alpha$, put α in cell M of A comma a , for every terminal a in FIRST of α . If ϵ is in FIRST of α , also put α in M of A comma b , for every b in FOLLOW of A . Okay so here's the key definition, this is important, this will be on the midterm: a grammar is LL(1) if and only if no cell ends up with two productions.

5:07 Equivalently, for any two alternatives A goes to alpha or beta, their FIRST sets must be disjoint, and if beta can be empty, FIRST of alpha must not overlap FOLLOW of A. Once you have the table, the parser itself is trivial. You keep a stack that starts with the start symbol and the end marker. If the top of the stack is a terminal, it must match the lookahead: pop it and advance the input.

5:18 If the top is a nonterminal A, look up M of A comma lookahead, pop A, and push the right-hand side in reverse. If the cell is empty, that's a syntax error. If the stack and the input are both at dollar, you accept. Everybody with me? Okay. Let's trace id plus id times id. Stack starts E dollar; lookahead is id, so we replace E with T E prime, then T with F T prime, then F with id, match, and keep going.

5:30 Okay, one more transformation you'll need, uh, left factoring. Suppose A goes to alpha beta one, or alpha beta two, with the same prefix alpha. One token of lookahead can't tell those apart, so we factor the prefix: A goes to alpha A prime, and A prime goes to beta one or beta two.

5:38 The classic example is if then else. Both productions start with if expr then stmt, so we factor that out and decide about the else later.

5:41 So for the dangling else, does the factoring actually fix the ambiguity?

5:43 No, and that's exactly the point. Factoring only delays the decision. The dangling else is genuinely ambiguous, and no amount of factoring makes it LL(1), so most compilers resolve it by always matching the nearest if. That's a general lesson. LL(1) is simple and fast, but some languages aren't LL(k) for any k, which is why we'll look at bottom-up parsing next.

5:52 Last thing, and then we're done: what happens when the table cell is empty and you hit an error. You don't want to stop at the first mistake, right. The simplest strategy is panic mode: discard input tokens until you reach a synchronizing token, then pop the nonterminal and continue.

5:59 A good choice of synchronizing set for A is FOLLOW of A, because after skipping to something in FOLLOW of A you can pretend A was parsed and move on. Report the error once, don't cascade, and keep parsing so the user sees more than one problem per compile. Next time we'll do bottom-up parsing, shift-reduce, and LR tables. Read Dragon Book sections four point four and four point five.